

ESCOLA MINEIRA DE ENGENHARIA

CIÊNCIA DA COMPUTAÇÃO

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES I

CAPÍTULO 3

ARITMÉTICA COMPUTACIONAL

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES I



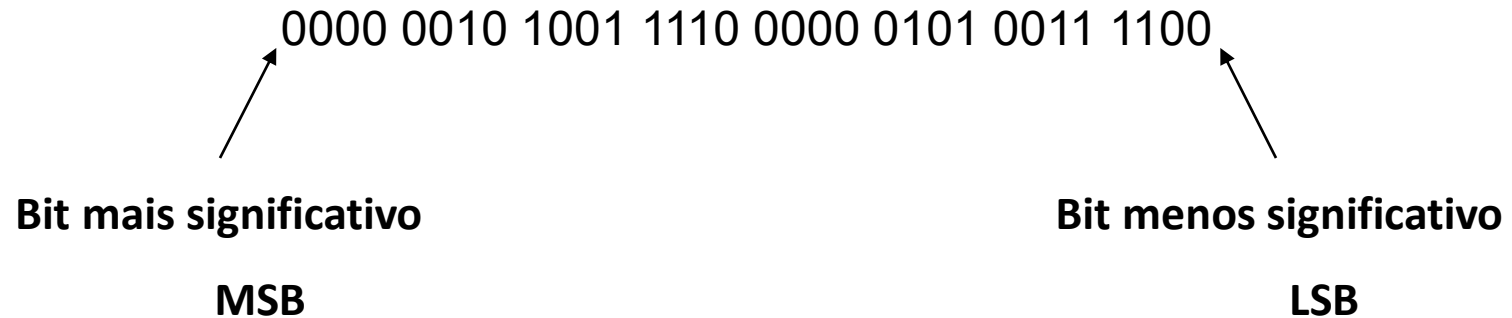
Aritmética dos Computadores

- As palavras dos computadores são compostas por bits
- Questões importantes sobre a representação de números nos computadores:
 - Como diferenciar números positivos e negativos?
 - Como representar números reais?
 - Qual o maior número que pode ser representado em uma palavra do computador?
 - O que acontece se uma operação gera um resultado maior do que o que pode ser armazenado pela palavra do computador?
- As duas últimas perguntas são extremamente importantes para os programadores.



Aritmética dos Computadores

- Números com sinal e sem sinal



- O bit mais significativo do número determina o seu sinal
- MSB = 0 → número positivo
- MSB = 1 → número negativo



Aritmética dos Computadores

- A quase totalidade dos computadores modernos utiliza o sistema de complemento a 2:
- O resultado da operação traz automaticamente o valor do bit de sinal → hardware mais simplificado.
- Se a palavra do computador possui 32 bits:

0000 ... 0000 → 0

0000 ... 0001 → 1

...

0111 ... 1111 → 2.147.483.647 → $2^{31} - 1$

1000 ... 0000 → -2.147.483.648 → -2^{31}

1000 ... 0001 → -2.147.483.647

...

1111 ... 1101 → -3

1111 ... 1110 → -2

1111 ... 1111 → -1



Aritmética dos Computadores

- Overflow → estouro da capacidade de armazenamento da palavra.
- No caso de números com sinal, ocorre quando:
 - Somamos dois números positivos e encontramos resultado negativo:
Ex. (com 4 bits):
 $+5 \rightarrow 0101$
 $+4 \rightarrow 0100$
 $+9 \rightarrow 1001 \rightarrow \text{ERRO: este número é } -7$
 - Somamos dois números negativos e encontramos resultado positivo:
Ex. (com 4 bits):
 $-5 \rightarrow 1011$
 $-4 \rightarrow 1100$
 $-9 \rightarrow 0111 \rightarrow \text{ERRO: este número é } +7$
- A ocorrência de overflow é detectada e sinalizada pelo hardware da máquina, através de uma EXCEÇÃO ou INTERRUPÇÃO.
- O tratamento deste erro é de responsabilidade do SISTEMA OPERACIONAL.

- Representação de números negativos em complemento a 2:
 - Negar todos os bits do número: $0101 \rightarrow 1010$
 - Somar 1 ao número negado:

	1
1010	+
1011	

+ 5
- 5
- Regra para extensão do sinal:
 - Se um número de 16 bits é armazenado em um registrador de 32 bits, o bit mais significativo do número é repetido até o 32º bit do registrador.
 - O número 2 em 16 bits é: $0000\ 0000\ 0000\ 0010$
 - Em 32 bits é: $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010$



Aritmética dos Computadores

- Veremos versões de algumas instruções do MIPS para números sem sinal:
 - addu → **add unsigned**
 - addiu → **add immediate unsigned**
 - subu → **sub unsigned** (não gera overflow)
 - sltu → **set on less than unsigned**

Ex.: suponha que o valor abaixo esteja no reg. \$s0

• 1111 1111 1111 1111 1111 1111 1111 1111

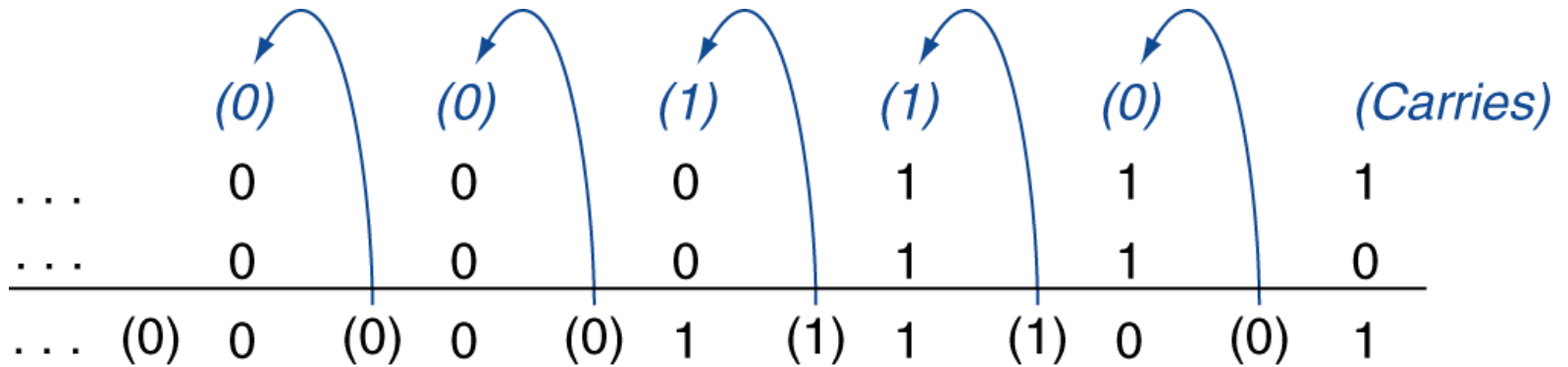
e que \$s1 tenha

• 0000 0000 0000 0000 0000 0000 0000 0010

- Quais serão os resultados das operações
 - slt \$t0, \$s0, \$s1 → \$t0 = ?
 - sltu \$t1, \$s0, \$s1 → \$t1 = ?

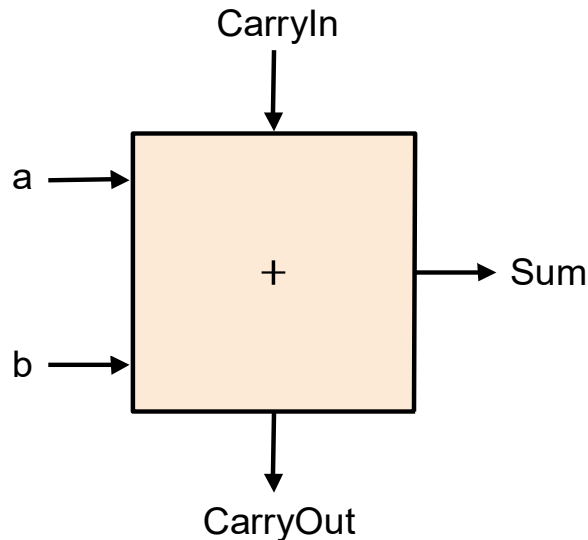
Adição

- Adição de inteiros:



Hardware

- As operações de soma e subtração envolvem 5 “entidades”:
 - 1º bit
 - 2º bit
 - O “vem-um” da soma anterior (CARRY- IN)
 - O resultado da operação (SUM)
 - O “vai-um” para a próxima soma (CARRY-OUT)



“Caixa preta”:
Somador completo de 1 bit.

Operações Lógicas

- Qualquer computador deve ser capaz de operar em bits individuais de uma palavra.

Ex.: `ori   $s0, $t0, 0x00D7` - operação “ou” bit a bit

\$to 0000 0000 0000 0000 0010 1101 0010 1100

0x00D7 0000 0000 1101 0111

\$s0 0000 0000 0000 0000 0010 1101 1111 1111

- O hardware executa estas instruções com o intuito de facilitar a colocação ou extração de bits dentro das palavras.

Operações Lógicas

- As instruções de “shifts” (deslocamentos) deslocam todos os bits de uma palavra para a esquerda ou para a direita.
- Os bits que ficam vazios são preenchidos com 0.

Ex.: \$s0 0000 0000 0000 0000 0000 0000 1100 0001

 sll \$t0, \$s0, 8 sll = SHIFT LEFT LOGICAL

 \$t0 0000 0000 0000 0000 1100 0001 0000 0000

Operações Lógicas

- Da mesma forma,

\$s3	0000 0000 0000 1111 0000 0000 1100 0001
srl \$t5, \$s3, 10	srl = SHIFT RIGHT LOGICAL
\$t5	0000 0000 0000 0000 0000 0011 1100 0000

- Durante a operação de deslocamento à direita os bits menos significativos do registrador são jogados fora.

Operações Lógicas

- Ex.: $AUX = 4 * i;$

Se $i = 7$	0000 0000 0000 0000 0000 0000 0000 0111
4	0000 0000 0000 0000 0000 0000 0000 0100
$4*i$	0000 0000 0000 0000 0000 0000 0001 1100

A multiplicação por 4
desloca o número 2
bits à esquerda

- Ex.: $AUX = 8 * i;$

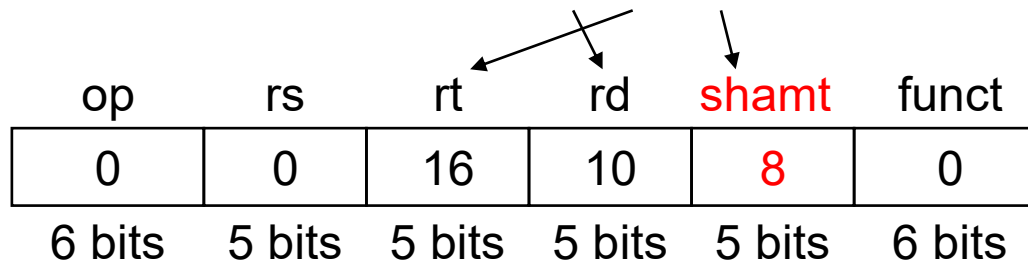
Se $i = 7$	0000 0000 0000 0000 0000 0000 0000 0111
8	0000 0000 0000 0000 0000 0000 0000 1000
$8*i$	0000 0000 0000 0000 0000 0000 0011 1000

A multiplicação por 8
desloca o número 3
bits à esquerda

- A multiplicação por 2^n desloca o número "n" bits à esquerda!

Operações Lógicas

- Por que utilizar o deslocamento à esquerda ao invés da multiplicação?
- Do que vimos até agora, onde poderíamos utilizar este recurso?
- Como fazer um deslocamento em linguagem de alto nível?
- Formato das instruções “shift”
- `sll $t2, $s0, 8`



- `sll` →
- O campo “SHAMT” significa “Shift Amount” (quantidade a deslocar).
- Como possui 5 bits, permite deslocamentos de até $2^5 = 32$ bits.

Operações Lógicas

- Operação AND bit a bit

\$t2 0000 0000 0111 0000 0000 1101 0000 0000

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

- and \$t0, \$t1, \$t2 \$t0 0000 0000 0000 0000 0000 1100 0000 0000

- A operação AND pode ser utilizada para verificar o valor de bits individuais em determinado registrador.

- Operação OR bit a bit

\$t2 0000 0000 0111 0000 0000 1101 0000 0000

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

- or \$t0, \$t1, \$t2 \$t0 0000 0000 0111 0000 0011 1101 0000 0000

- A operação OR pode ser utilizada para colocar 1 em determinados bits.

Operações Lógicas

- `andi $t0, $s1, 240` → `andi` = and immediate

`$s1` → 0000 0101 0001 0000 0011 0001 0111 1111

`240` → 0000 0000 1111 0000

`$t0` → 0000 0101 0001 0000 0000 0000 0111 0000

- `andi` não altera os 16 bits mais significativos!
- `andi $t0, $s1, 0x2B7F`

`$s1` → 0000 0101 0001 0000 0011 0001 0111 1111

`0x2B7F` → 0010 1011 0111 1111

`$t0` → 0000 0101 0001 0000 0010 0001 0111 1111

Operações Lógicas

- ori \$t0, \$s1, 240 → ori = or immediate

\$s1 → 0000 0101 0001 0000 0011 0001 0111 1111

240 → 0000 0000 1111 0000

\$t0 → 0000 0101 0001 0000 0011 0001 1111 1111

- ori também não altera os 16 bits mais significativos!
- ori \$t0, \$s1, 0x2B7F

\$s1 → 0000 0101 0001 0000 0011 0001 0111 1111

0x2B7F → 0010 1011 0111 1111

\$t0 → 0000 0101 0001 0000 0011 1011 0111 1111



Resumo das Instruções Vistas

- Aritméticas:

add \$s1,\$s2,\$s3 $\$s1 = \$s2 + \$s3$

addi \$s1,\$s2,4 $\$s1 = \$s2 + 4$

addu \$s1,\$s2,\$s3 $\$s1 = \$s2 + \$s3$ sem geração de overflow

addiu \$s1,\$s2,10 $\$s1 = \$s2 + 10$ sem verificação de sinal

sub \$s1,\$s2,\$s3 $\$s1 = \$s2 - \$s3$

subu \$s1,\$s2,\$s3 $\$s1 = \$s2 - \$s3$ sem geração de overflow

mul \$s1,\$s2,\$s3 $\$s1 = \$s2 * \$s3$

- Lógicas:

sll \$t0,\$s1,4 $\$t0 = \$s1 \ll 4$

srl \$t0,\$t1,4 $\$t0 = \$t1 \gg 4$

and \$t0,\$s1,\$s2 $\$t0 = \$s1 \& \$s2$

or \$t0,\$s2,\$s2 $\$t0 = \$s1 | \$s2$

andi \$t0,\$s1, 20 $\$t0 = \$s1 \& 20$

ori \$t0,\$s1, 20 $\$t0 = \$s1 | 20$



Resumo das Instruções Vistas

- Transferência de dados:

lw \$s1,100(\$s2) \$s1 = Memory[\$s2+100]

sw \$s1,100(\$s2) Memory[\$s2+100] = \$s1

lui \$s1,0x1000 \$s1 = 0x10000000

- Desvios:

beq \$s4,\$s5,L1 Próxima instr. está no Label se \$s4 = \$s5

bne \$s4,\$s5,L1 Próxima instr. está no Label se \$s4 ≠ \$s5

j Label Próxima instr. está no Label

slt \$t0,\$s0,\$s1 \$t0=1 se \$s0 < \$s1

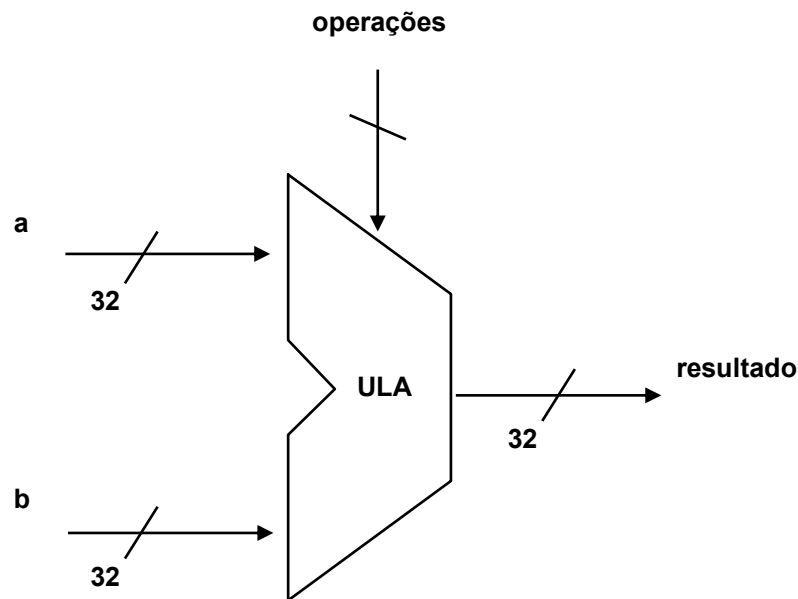
 \$t0=0 se \$s0 ≥ \$s1

sltu \$t0,\$s0,\$s1 \$t0=1 se \$s0 < \$s1

 \$t0=0 se \$s0 ≥ \$s1

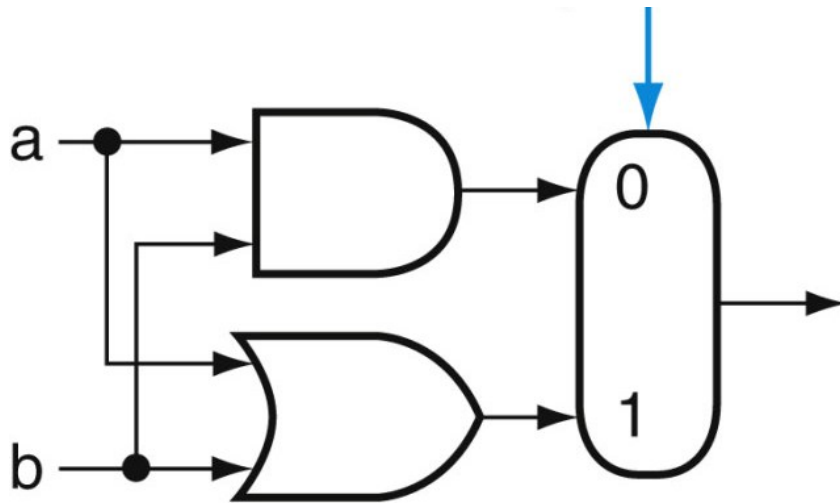
Construção da ULA

- A Unidade Lógica Aritmética é o dispositivo que realiza as operações matemáticas, como adição, subtração, OR, AND e shifts.
- Os processadores com palavras de 32 bits possuem ULAs também com 32 bits.



Construção da ULA: 1 bit

- A construção de uma ULA de 32 bits pode ser feita construindo-se uma ULA de 1 bit e conectando-se 32 destas unidades.
- As operações lógicas bit a bit são as mais simples de realizar porque elas são executadas diretamente pelas portas lógicas já estudadas.



Multiplexador:

Operation = 0 $\Rightarrow$ a E b

Operation = 1 $\Rightarrow$ a OU b

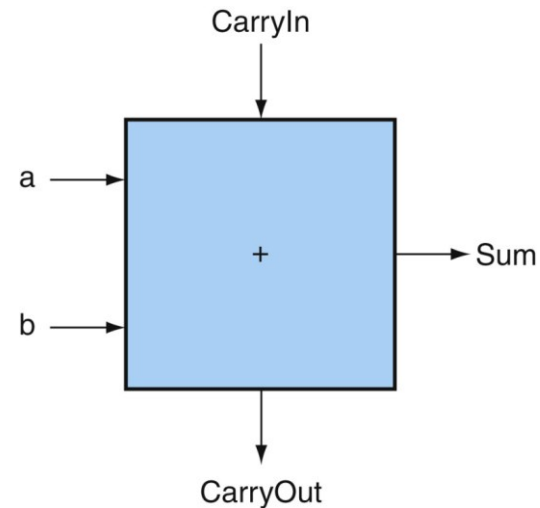
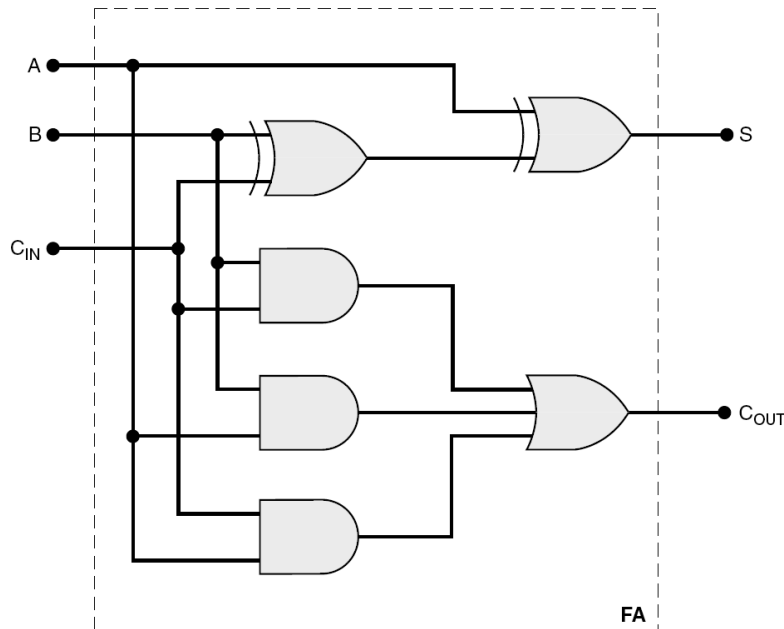
Em Assembly:

```
and $t0, $s0, $s1
```

```
or  $t0, $s0, $s1
```

Construção da ULA: 1 bit

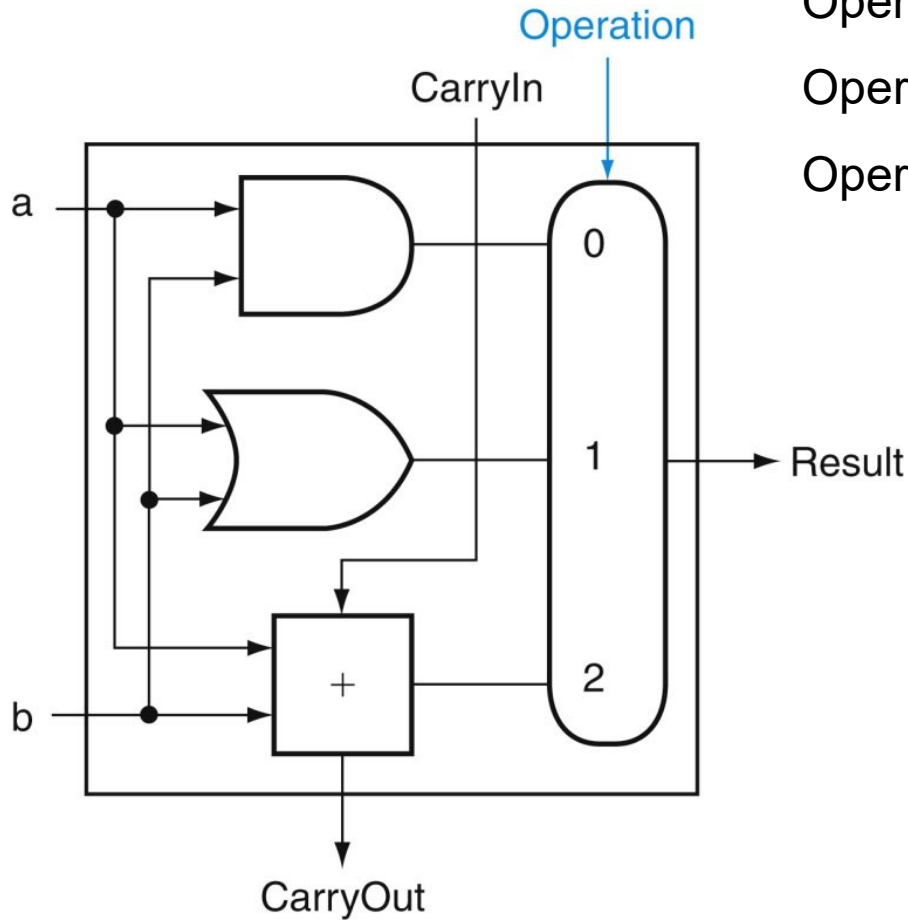
- A próxima função a ser inserida é a adição.
- O somador completo de 1 bit será utilizado para fazer esta operação.



$$C_{Out} = a b + a c_{in} + b c_{in}$$
$$S = a \oplus b \oplus c_{in}$$

Construção da ULA: 1 bit

- Unificando Soma, And e Or:



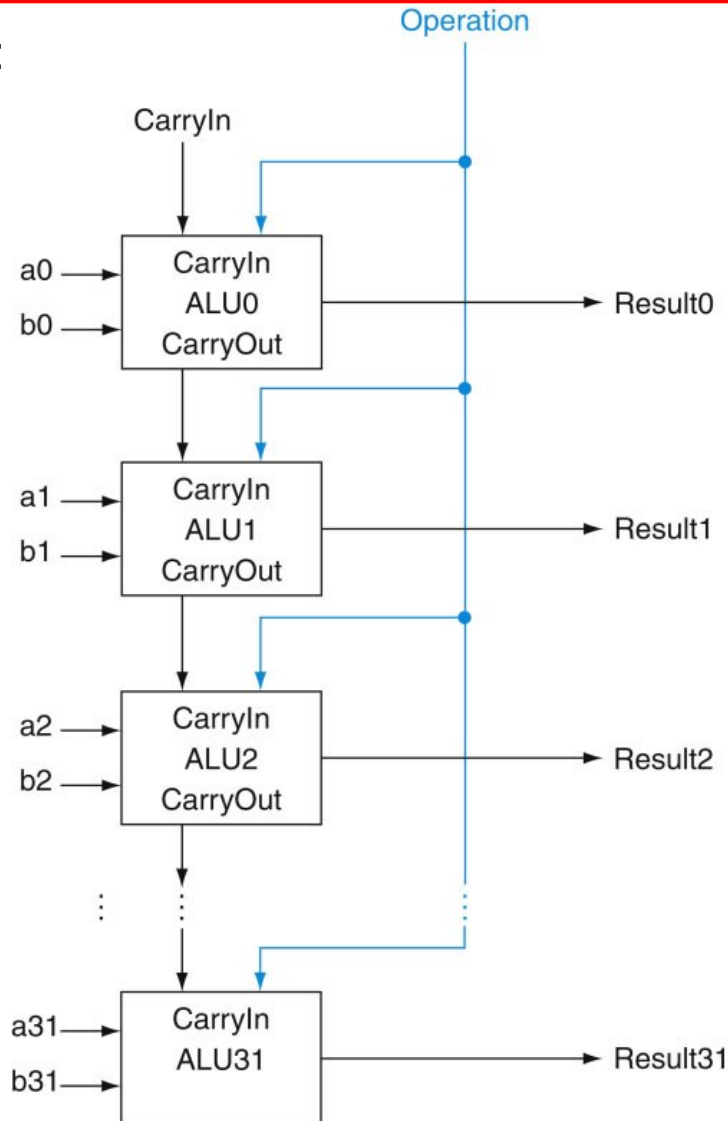
Operation = 0 $\Rightarrow$ a E b

Operation = 1 $\Rightarrow$ a OU b

Operation = 2 $\Rightarrow$ a + b

Construção da ULA: 1 bit

- ULA de 32 bits:



O carry-out gerado no primeiro bloco pode se propagar por toda a extensão do somador e atingir o bit mais significativo.



SOMADOR DE CARRY PROPAGADO

Construção da ULA

- Subtração
- Já vimos que a subtração é realizada através da técnica de complemento de 2: nega-se todos os bits do subtraendo e adiciona-se 1 ao resultado.
- Para realizar a inversão dos bits acrescenta-se à entrada do somador um multiplexador 2 : 1 que escolhe entre b e b-barrado.

$\text{Binvert} = 0 \Rightarrow b$

$\text{Binvert} = 1 \Rightarrow b\text{-barrado}$

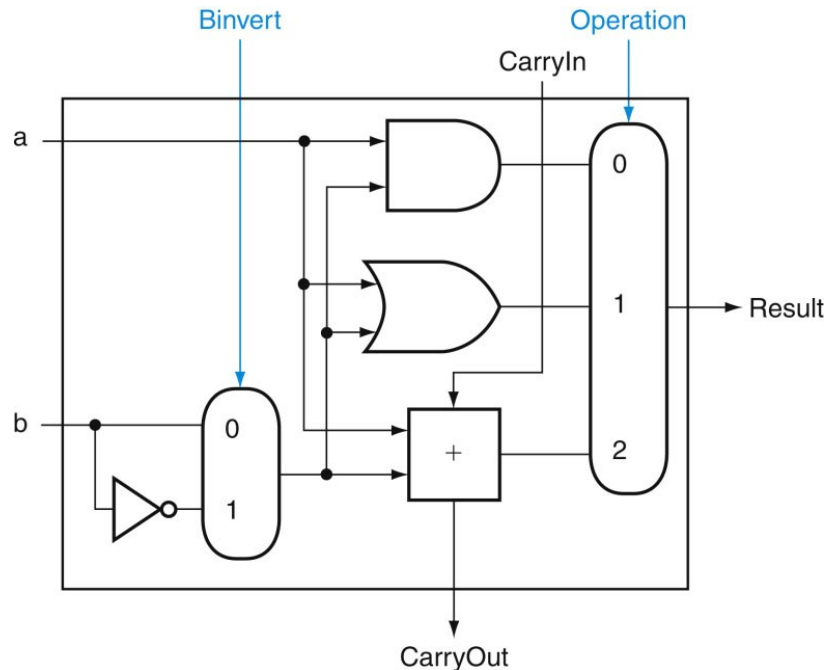
$\text{Operation} = 0 \Rightarrow a \text{ E } b$

$\text{Operation} = 1 \Rightarrow a \text{ OU } b$

$\text{Operation} = 2 \Rightarrow a + b$

$\text{CarryIn} = 0 \Rightarrow \text{Soma}$

$\text{CarryIn} = 1 \Rightarrow \text{Subtração}$



Construção da ULA

- Subtração
- Como somar 1 ao resultado de b-barrado?
- O carry-in do 1º somador é inútil para a soma, mas pode ser utilizado para fazer o complemento de 2 em um subtração.

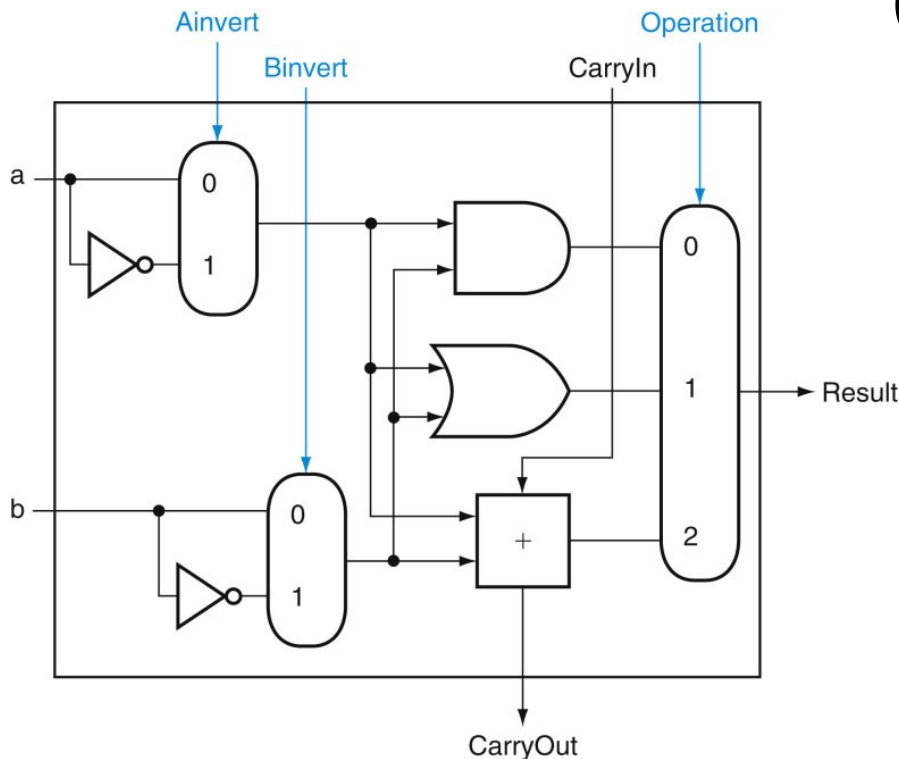
$$a + \bar{b} + 1 = a + (\bar{b} + 1) = a + (-b) = a - b$$

- A simplicidade do hardware foi uma das razões para se escolher a técnica do complemento de 2.

Construção da ULA

- Uma ALU no MIPS também precisa de uma função NOR. Em vez de acrescentar uma porta separada para NOR, podemos reutilizar grande parte do hardware já existente na ALU, como fizemos para a subtração.

$$\overline{(a + b)}(\text{NOR}) = \bar{a}.\bar{b}$$



Ainvert = 0 $\Rightarrow$ a

Ainvert = 1 $\Rightarrow$ a-barrado

Binvert = 0 $\Rightarrow$ b

Binvert = 1 $\Rightarrow$ b-barrado

CarryIn = 0 $\Rightarrow$ Soma

CarryIn = 1 $\Rightarrow$ Subtração

Operation = 0 $\Rightarrow$ a E b

Operation = 1 $\Rightarrow$ a OU b

Operation = 2 $\Rightarrow$ a + b

Instrução slt

- `slt $t0, $s0, $s1`

No exemplo, “set on less than” produz:

- $\$t0 = 1$ se o registrador $\$s0 < \$s1$
- $\$t0 = 0$ se o registrador $\$s0 \geq \$s1$
- **CONSEQUÊNCIA:** a instrução `slt` altera somente o 1º bit do registrador de destino, deixando todos os demais em zero. Ex.:

Se $\$s0 < \$s1 \Rightarrow \$t0 = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001$ 

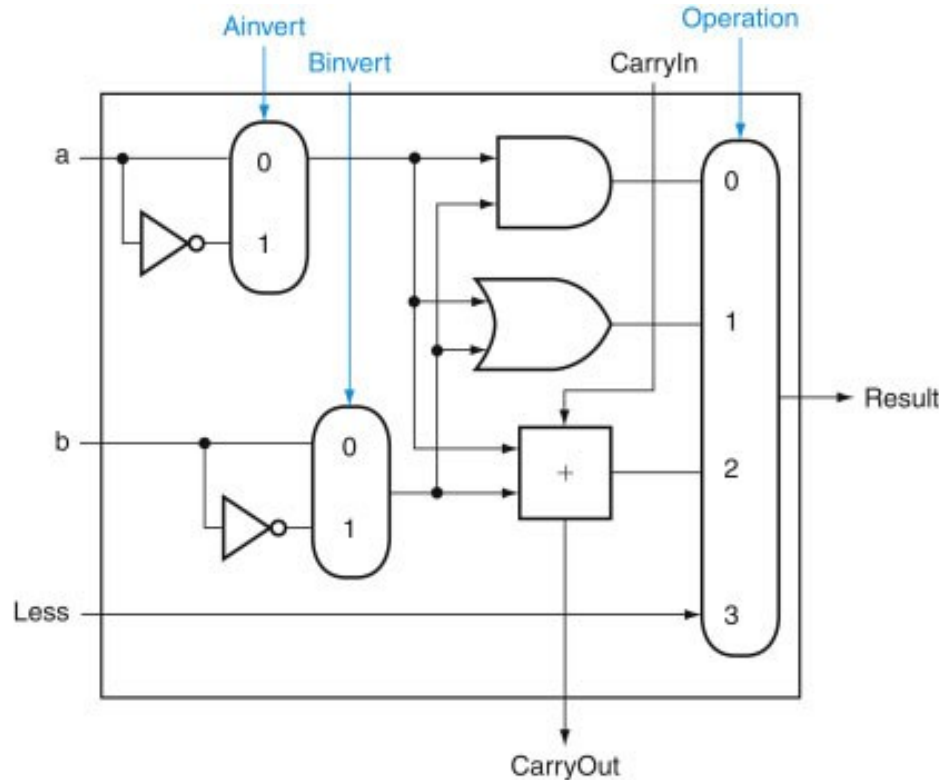
Se $\$s0 \geq \$s1 \Rightarrow \$t0 = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$ 

- A partir da descrição de `slt` anterior, temos de conectar 0 à entrada Less para os 31 bits superiores da ALU, pois esses bits sempre serão 0. O que falta considerar é como comparar e definir o valor do bit menos significativo para instruções “set on less than”.

Instrução slt

- Para executar slt primeiro precisamos expandir o multiplexador de saída da ULA

Entrada “less”
utilizada
somente na
instrução “slt”



$Ainvert = 0 \Rightarrow a$

$Ainvert = 1 \Rightarrow a\text{-barrado}$

$Binvert = 0 \Rightarrow b$

$Binvert = 1 \Rightarrow b\text{-barrado}$

$CarryIn = 0 \Rightarrow \text{Soma}$

$CarryIn = 1 \Rightarrow \text{Subtração}$

$Less = 1 \Rightarrow \text{slt}$

$Operation = 0 \Rightarrow a \text{ E } b$

$Operation = 1 \Rightarrow a \text{ OU } b$

$Operation = 2 \Rightarrow a + b$

Instrução slt

- Como a instrução slt modifica somente o bit menos significativo do resultado, todos os demais bits devem ser iguais a zero.

- Como verificar se um número A é menor que outro número B?

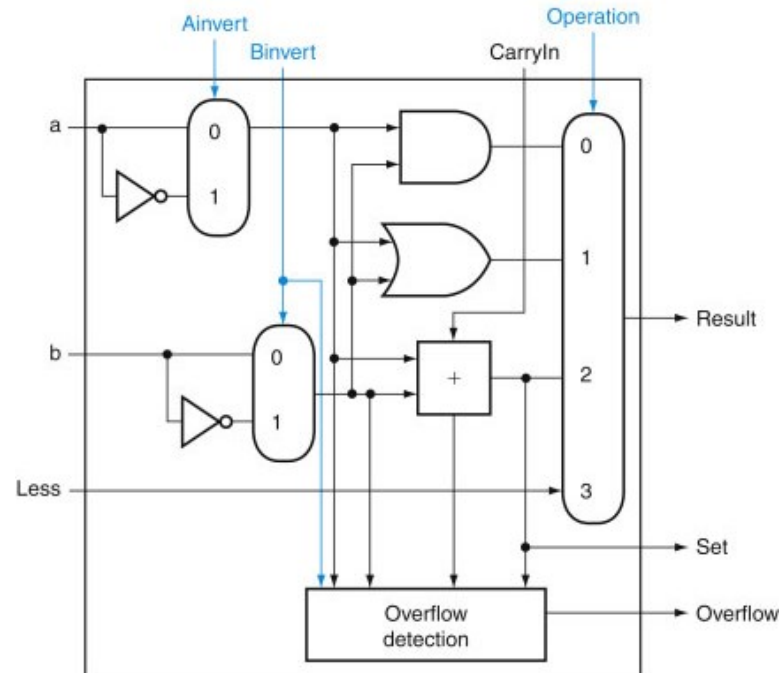
Se $A > B$, $A - B$ dá resultado positivo, o bit de sinal do resultado = 0

Se $A < B$, $A - B$ dá resultado negativo, o bit de sinal do resultado = 1

- CONSEQUÊNCIA: o primeiro bit do resultado da operação “slt” pode ser obtido diretamente do bit de sinal gerado de uma operação de subtração.
- Este bit é obtido do último bloco da nossa ULA.

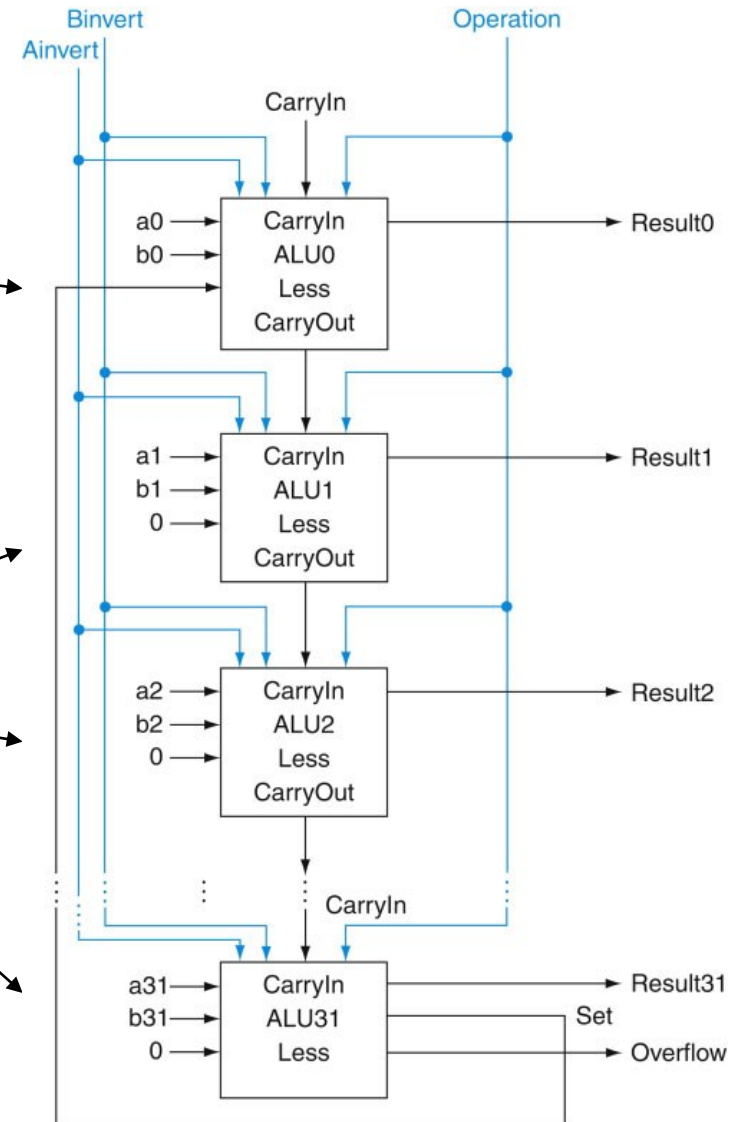
Instrução slt

- Precisamos de uma nova ALU de 1 bit, para o MSB com um bit de saída extra: a saída do somador.
- Nova linha de saída do somador: Set (usada apenas para slt).
- Como precisamos de uma ALU especial para MSB, acrescentamos a lógica de detecção de overflow, pois também está associada a esse bit.



Instrução slt

- Assim, a saída “set” do último bloco é conectada diretamente à entrada “less” do primeiro bloco.
- Todas as outras entradas “less” ficam sempre com o valor 0.



Condensando Entradas

- Observe que toda vez que quisermos que a ALU subtraia, colocamos CarryIn e Binvert em 1.
- Para adições ou operações lógicas, queremos que as duas linhas de controle sejam 0.
- Portanto, podemos simplificar o controle da ALU combinando CarryIn e Binvert a uma única linha de controle, chamada Bnegate.

Instruções de Desvio

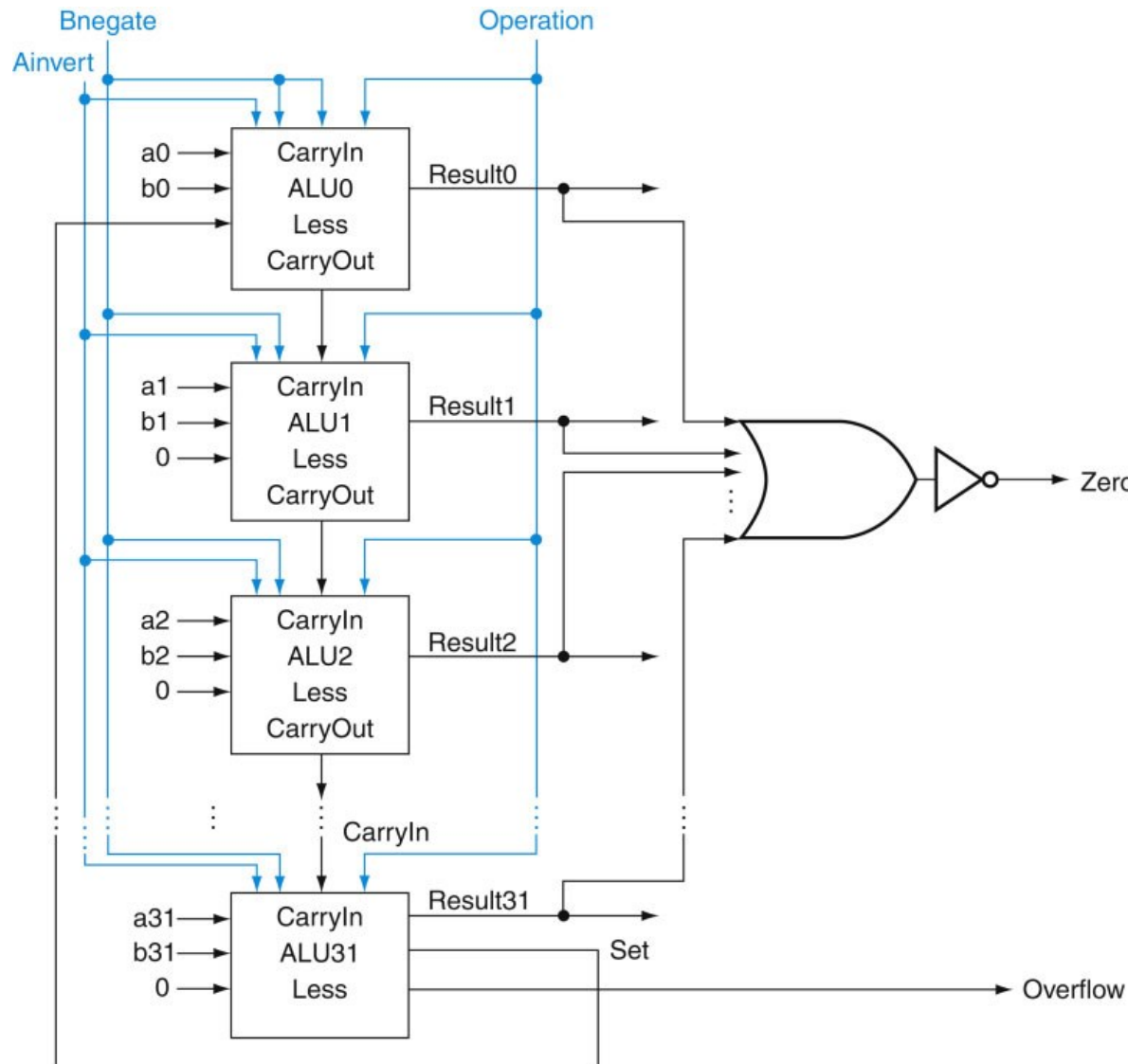
- As últimas operações necessárias à nossa ULA testam a igualdade de dois números e desviam caso sejam iguais (beq) ou caso sejam diferentes (bne).
- A maneira mais simples de testar se dois números são iguais é subtrair e verificar se o resultado é igual ou diferente de 0.
- Se $A - B = 0 \Rightarrow A = B$.
- Para verificar se a saída de nossa ULA é igual a zero ou não basta passar todas as saídas pela função OU seguida de um inversor.

Instruções de Desvio

- A linha de controle “Operation” controla um multiplexador de 4 entradas, sendo na verdade 2 linhas de controle: OP0 e OP1.
- A associação das linhas Ainvert, Bnegado com as duas linhas OP0 e OP1 (nessa ordem) permite realizar 6 operações:

Controle	Função
0000	AND
0001	OR
0010	ADIÇÃO
0110	SUBTRAÇÃO
0111	slt
1100	NOR

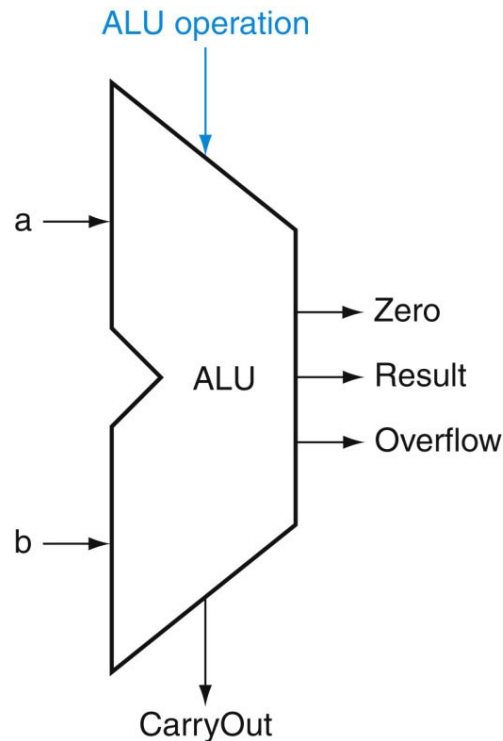
Projeto Final da ULA



Nota: a saída
“zero” é 1
quando o
resultado é 0

Representação da ULA

- Agora que sabemos o que existe dentro de uma ULA podemos adotar o símbolo universal para sua representação.



Representação da ULA

- Podemos integrar uma ALU para suportar o conjunto de instruções do MIPS
- Ideias chave:
 - usar multiplexador para selecionar na saída a operação desejada;
 - fazer subtrações usando complemento de dois;
 - replicar a ULA de 1-bit para produzir uma ULA de 32-bits.
- Pontos importantes sobre hardware:
 - todas as portas estão sempre trabalhando;
 - a velocidade da porta é afetada pelo seu número de entradas;

Controle da ULA

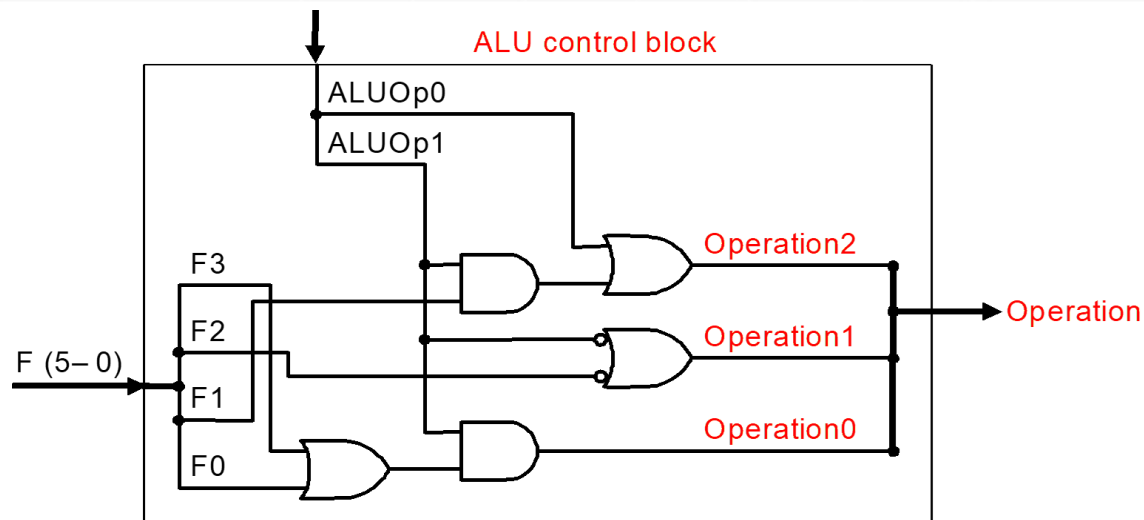
- Controles:

opcode	ALUOp	Operação	funct	Ação da ALU desejada	ALU control
lw (35)	00	load word	Não há	add	0010
sw (43)	00	store word	Não há	add	0010
beq (4)	01	branch on equal	Não há	subtract	0110
Tipo R	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

Controle da ULA

- Implementação por lógica combinacional

ALUOp		Campo funct						Operação
ALUOp1	ALUOp2	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111



Multiplicação

- Acabamos de implementar uma ULA que realiza as operações:
 - and bit a bit
 - or bit a bit
 - soma
 - subtração
 - slt → set on less than
 - beq → branch if equal
 - bne → branch if not equal
- Como o computador realiza multiplicação e divisão?
 - Obtidas através de deslocamentos e somas.
 - Gastam mais tempo e ocupam mais espaço de memória do que a adição e subtração.
 - Veremos 1ª versão, baseada no algoritmo aprendido no ensino fundamental.

Multiplicação

- Exemplo:

$$\begin{array}{r} 1000 \rightarrow \text{multiplicando} \\ \times 1001 \rightarrow \text{multiplicador} \\ \hline 1000 \\ 0000 \\ 0000 \\ 1000 \\ \hline 1001000 \end{array}$$

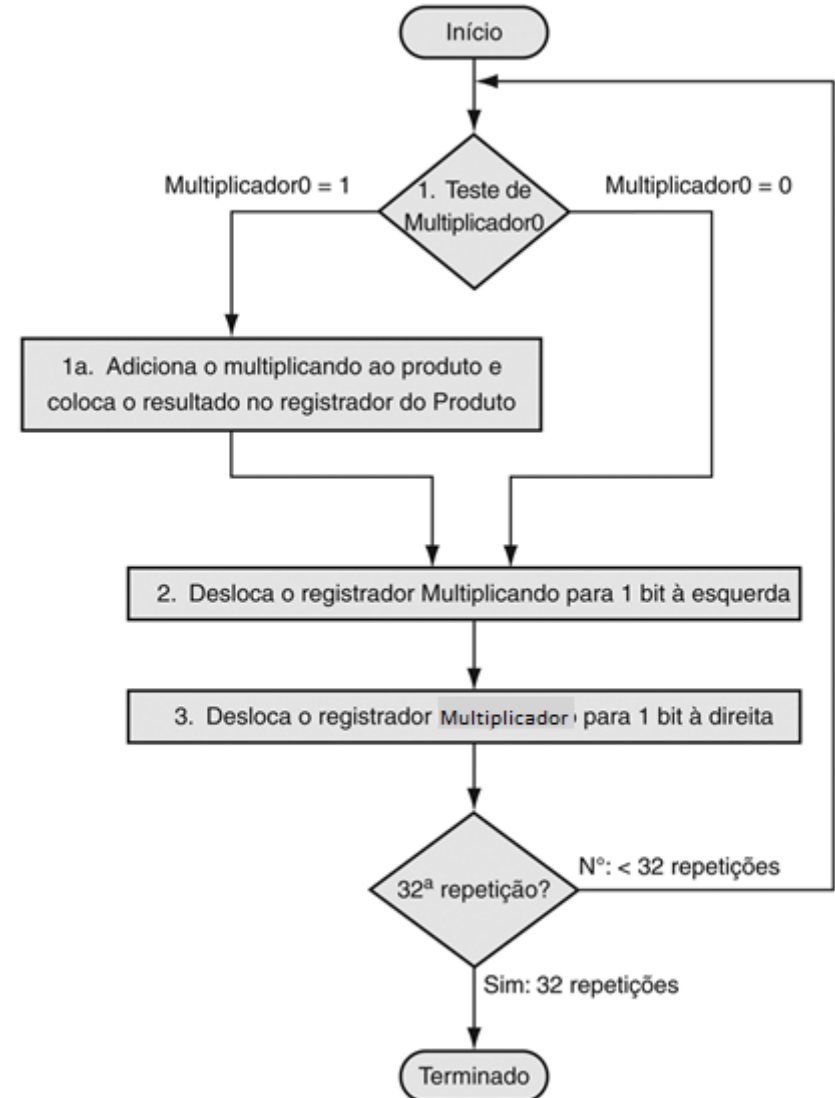
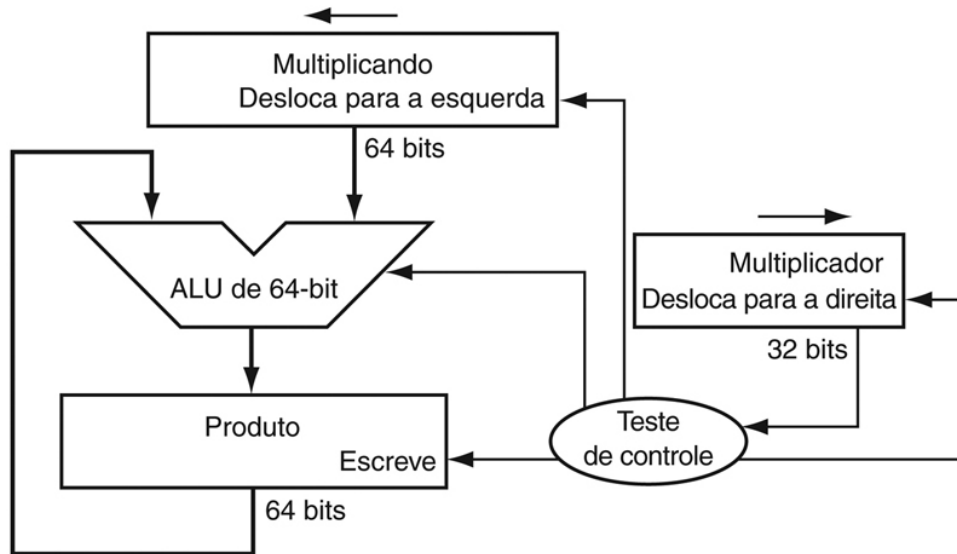
- Considerações:
 - O número de dígitos do produto é muito maior que o número de dígitos do multiplicando ou do multiplicador.
 - Multiplicando $\rightarrow$ m bits; Multiplicador $\rightarrow$ n bits.
 - Tamanho do produto = $m + n - 1$ bits.

Multiplicação

- Multiplicação de números negativos:
 - Converter para números positivos e multiplicar.
 - Ajustar o resultado.
 - Técnicas otimizadas → algoritmo de Booth

Multiplicação

- Algoritmo



Divisão

- Mais complexa do que a multiplicação.
- Oportunidade de se efetuar uma operação inválida: divisão por 0.

Ex.: 1.001.010 | 1.000 Divisor
 Dividendo

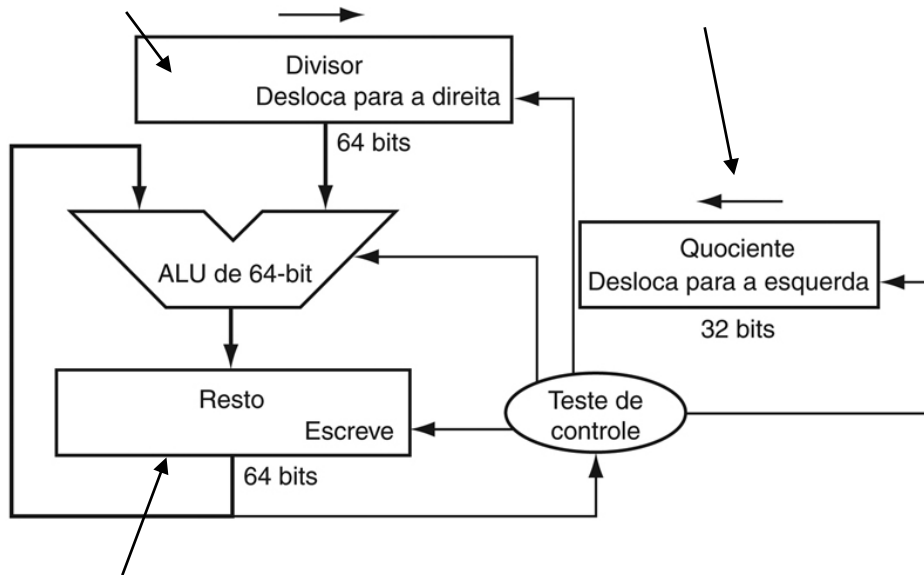
- A divisão envolve 4 entidades:
 - Dividendo
 - Divisor
 - Quociente
 - Resto: torna a divisão diferente da multiplicação
- $\text{Dividendo} = (\text{Quociente} \times \text{Divisor}) + \text{Resto}$

Divisão

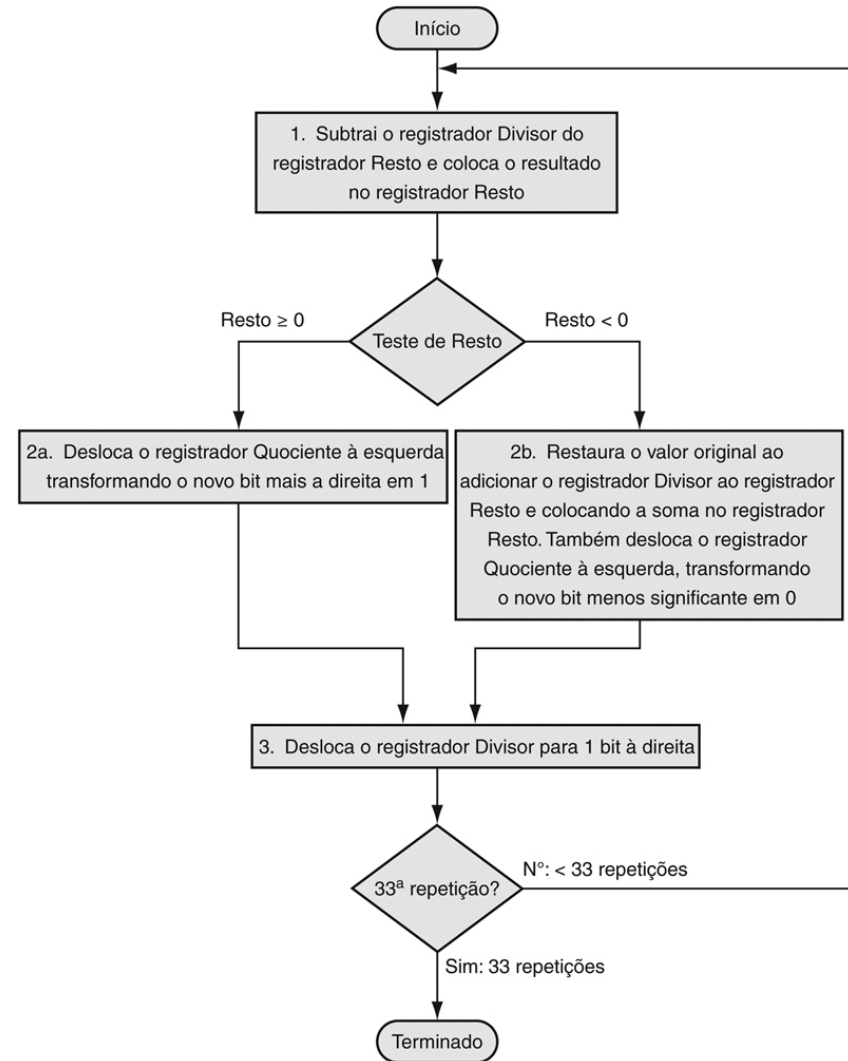
Algoritmo

Divisor inicializado nos 32 bits inferiores.

Inicializado em 0



O registrador do resto é inicializado com o dividendo nos 32 bits superiores.



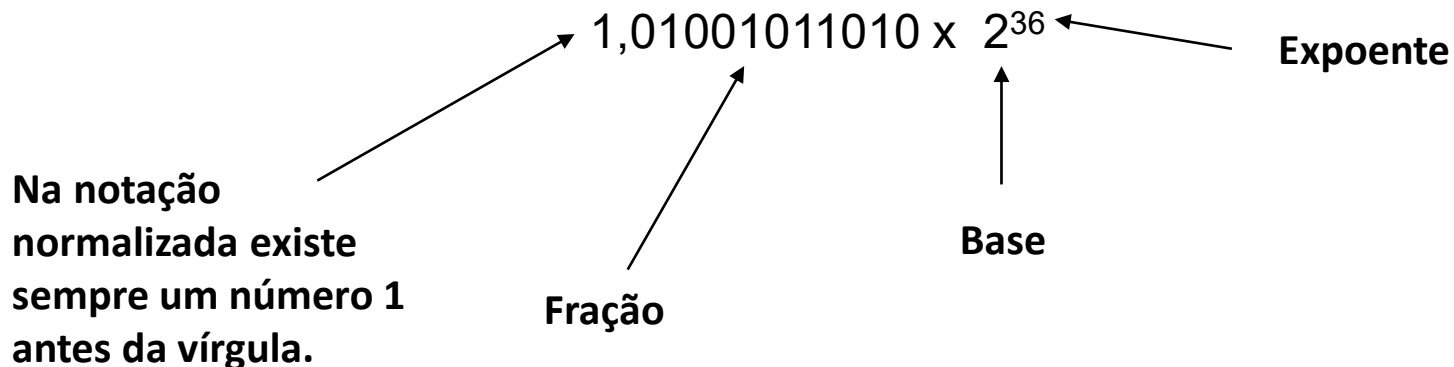


MIPS: Divisão e Multiplicação

- `mul $t0, $s0, $s1` → pseudo instrução
- `mul` é uma combinação das instruções
 - `mult $s0, $s1` → multiplica `$s0` por `$s1`
 - `mflo $t0` → move from lo
- `mflo` busca o resultado do registrador produto (32 bits inferiores)
- `div $t0, $s0, $s1` → pseudo instrução
- `div` é uma combinação das instruções.
 - `div $s0, $s1` → divide `$s0` por `$s1`
 - `mflo $t0` → move o quociente para `$t0`
- Para se obter o resto da divisão utiliza-se a instrução `mfhi`
 - `mfhi $t3` → move from hi

Ponto Flutuante

- Até o momento temos tratado com operandos inteiros.
- Entretanto, os computadores precisam de uma forma de representar:
 - Números fracionários: 3,14159265
 - Números muito pequenos: 0,000000001 ou 1×10^{-9}
 - Números muito grandes: 3.155.760.000 ou $3,15576 \times 10^9$
- Os números em ponto flutuante em binário são representados da forma normalizada:



Ponto Flutuante

- Definido pelo padrão IEEE Std 754-2008;
- Desenvolvido para resolver pendências sobre divergências de representação numérica;
- Portabilidade;
- Quase universalmente adotado;
- Gera Overflow/Underflow;

Precisão simples		Precisão dupla		Objeto representado
Expoente	Fração	Expoente	Fração	
0	0	0	0	0
0	não zero	0	não zero	$\pm$ número desnormalizado
1–254	qualquer coisa	1–2046	Anything	$\pm$ número ponto flutuante
255	0	2047	0	$\pm$ infinito
255	não zero	2047	não zero	Not a Number (NaN)

Ponto Flutuante

- O IEEE 754 até mesmo possui um símbolo para o resultado de operações inválidas, como $0/0$ ou a subtração entre infinito e infinito. Esse símbolo é NaN, de Not a Number (não é um número).
- A finalidade dos NaNs é permitir que os programadores adiem alguns testes e decisões para outro momento no programa, quando for conveniente.
- Os projetistas do IEEE 754 também queriam uma representação de ponto flutuante que pudesse ser facilmente processada por comparações de inteiros, especialmente para ordenação.
- Esse desejo é o motivo pelo qual o sinal está no bit mais significativo, permitindo um teste rápido de menor que, maior que ou igual a 0. (Isso é um pouco mais complicado do que uma ordenação simples de inteiros, pois essa notação é basicamente sinal e magnitude, em vez do complemento de dois.)

Ponto Flutuante

- Colocar o expoente antes do significando simplifica a ordenação dos números de ponto flutuante usando instruções de comparação de inteiros, pois os números com expoentes grandes são maiores do que os números com expoentes menores, desde que os dois expoentes tenham o mesmo sinal.
- Seguindo as diretrizes do IEEE, o comitê IEEE 754 foi modificado 20 anos depois do padrão para ver quais mudanças deveriam ser feitas ou se deveriam.
- O padrão revisado IEEE 754-2008 inclui quase todo o IEEE 754-1985 e acrescenta um formato de 16 bits (“meia precisão”) e um formato de 128 bits (“precisão quádrupla”). Nenhum hardware foi criado para dar suporte à precisão quádrupla, mas ele certamente virá.
- O padrão revisado também acrescenta aritmética de ponto flutuante, que os mainframes IBM implementaram.

46,6875

• 46:

- Passo 1: $46 / 2 = 23$ resto = 0
- Passo 2: $23 / 2 = 11$ resto = 1
- Passo 3: $11 / 2 = 5$ resto = 1
- Passo 4: $5 / 2 = 2$ resto = 1
- Passo 5: $2 / 2 = 1$ resto = 0
- Passo 6: $1 / 2 = 0$ resto = 1
- Resultado $46_{10} \Rightarrow 101110_2$



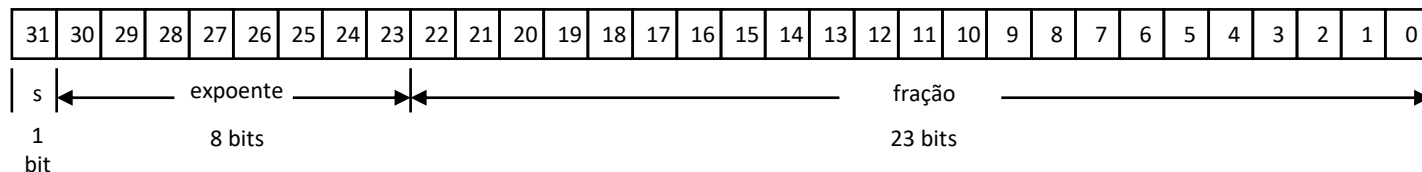
46,6875

• 0,6875:

- Passo 1: $0.6875 * 2 = 1.3750$ inteiro = 1
 - Passo 2: $0.3750 * 2 = 0.7500$ inteiro = 0
 - Passo 3: $0.7500 * 2 = 1.5000$ inteiro = 1
 - Passo 4: $0.5000 * 2 = 1.0000$ inteiro = 1
 - Passo 5: $0.0000 * 2 = 0.0000$ inteiro = 0
 - Resultado $0.6875_{10} \Rightarrow 0.10110_2$
-
- $46,6875_{10} = 101110,10110_2$

Ponto Flutuante

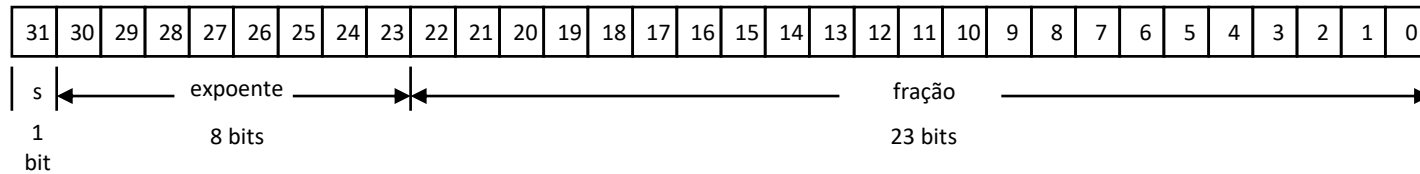
- Ex.: represente os seguintes números em binário utilizando a representação normalizada:
 - a) 101
 - b) 0,10111
 - c) 1101011,011
 - d) 100111×2^{-3}
- O armazenamento de um número em ponto flutuante no computador é realizado da seguinte forma:
 - Em precisão simples



Ponto Flutuante

- A precisão simples permite a representação de números (em tese) na faixa de:

$$2,0 \times 2^{-127} \text{ a } 2,0 \times 2^{128}$$



Forma Geral: $(-1)^s \cdot F \cdot 2^E$ onde

s - sinal

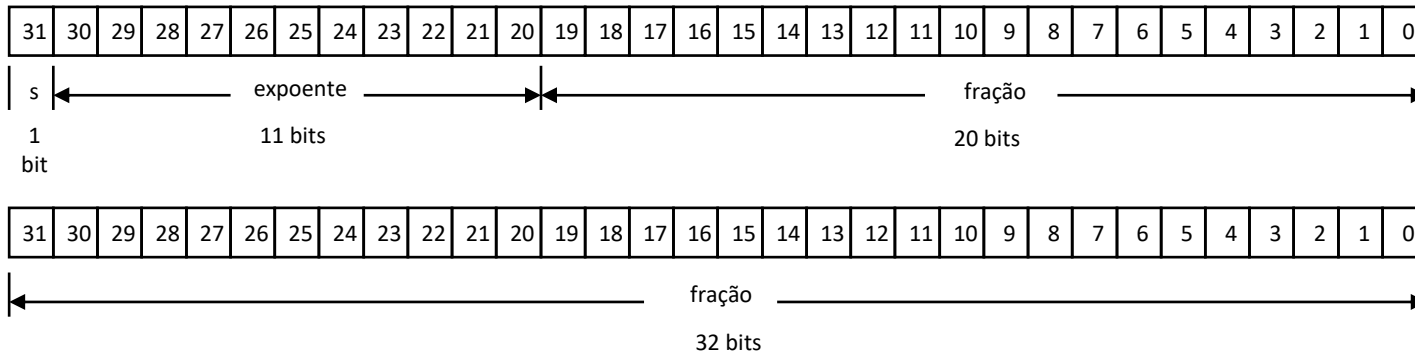
F - fração

E - Expoente

Ponto Flutuante

- A precisão dupla permite a representação de números (em tese) na faixa de:

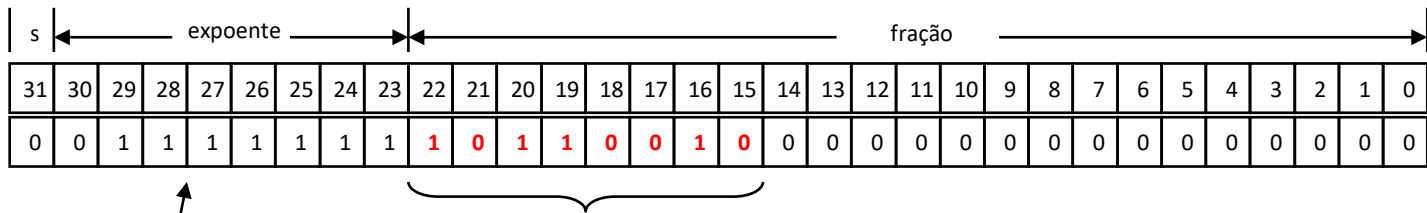
$$2,0 \times 2^{-1023} \text{ a } 2,0 \times 2^{1024}$$



Ponto Flutuante

- Como há sempre um bit 1 antes da vírgula na forma normalizada, este primeiro bit não é representado, ficando implícito.

Ex.: 1,10110010



Expoente:
representação
com “peso”

Somente os números
após a vírgula são
representados

Ponto Flutuante

- O cálculo do expoente é feito através da “representação com peso”.
- Em precisão simples: peso = 127
- Em precisão dupla: peso = 1023

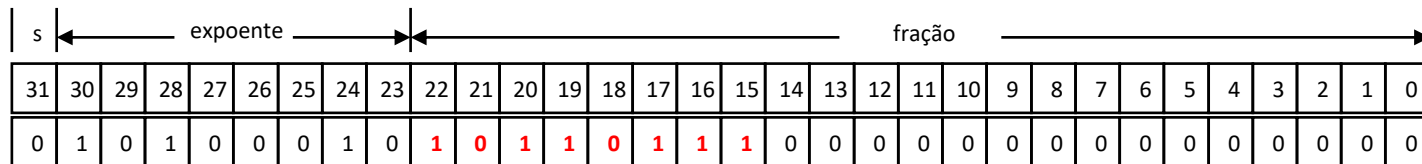
Ex.: representar o número $1,10110111 \times 2^{35}$ em precisão simples.

1º - número positivo bit de sinal = 0

2º - somente os números à direita da vírgula são representados

3º - cálculo do expoente a ser representado:

$$35 + 127 = 162 = 10100010_{(2)}$$

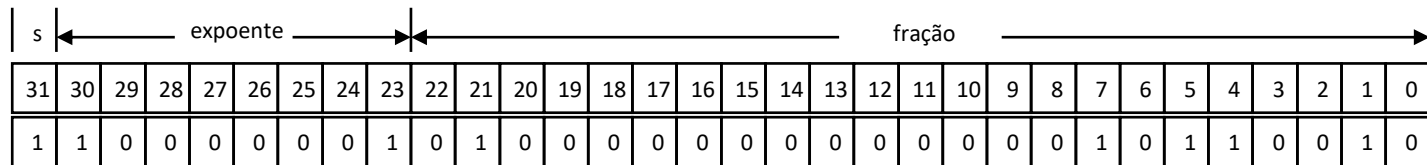


- O uso da representação com peso visa otimizar a ordenação de números em ponto flutuante.

Ponto Flutuante

Exemplos:

- Mostrar a representação binária do número 0,75 em precisão simples e dupla.
- Mostrar a representação binária do número $1001,001011110 \times 2^{-26}$ em precisão simples.
- Qual número decimal real corresponde à representação a seguir



Ponto Flutuante: Simples

- Expoentes 00000000 e 11111111 são reservados
- Menor valor para o expoente é: $00000001 = 1 - 127 = -126$
- Fração: 000...00 significando 1.0
- $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Maior Expoente: $11111110 = 254 - 127 = +127$
- Fração: 111...11 significando 2.0
- $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Ponto Flutuante: Duplo

- Expoentes 0000...00 e 1111...11 reservados.
- Menor valor para o expoente é 000000000001 = 1 – 1023 = –1022
- Fração: 000...00 significando 1.0
- $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$
- Maior valor para o expoente é 11111111110 = 2046 – 1023 = +1023
- Fração: 111...11 significando ≈ 2.0
- $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Ponto Flutuante: Precisão

- Precisão Relativa
- Todos os bits da mantissa são significativos!
- Single: aproximadamente 2^{-23}
- Equivalente a $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 7$ dígitos decimais de precisão
- Double: aproximadamente 2^{-52}
- Equivalente a $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ dígitos decimais de precisão

Ponto Flutuante: Adição

- Considere o exemplo decimal: $9.999 \times 10^1 + 1.610 \times 10^{-1}$

1. Alinhamento

- Deslocar o número com o menor expoente
- $9.999 \times 10^1 + 0.016 \times 10^1$

2. Somar frações

- $9.999 \times 10^1 + 0.016 \times 10^1 = 10.015 \times 10^1$

3. Normalizar o resultado e verificar se houve overflow/underflow

- 1.0015×10^2

4. Arredondar e renormalizar se necessário

- 1.002×10^2

Ponto Flutuante: Adição

- Considere a soma binária $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$ ($0.5 + -0.4375$)

1. Alinhamento

- Deslocar o número com o menor expoente
- $1.000_2 \times 2^{-1} + -0.1110_2 \times 2^{-1}$ ($0.5 + -0.4375$)

2. Somar mantissas

- $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$

3. Normalizar o resultado e verificar se houve overflow/underflow

- $1.000_2 \times 2^{-4}$

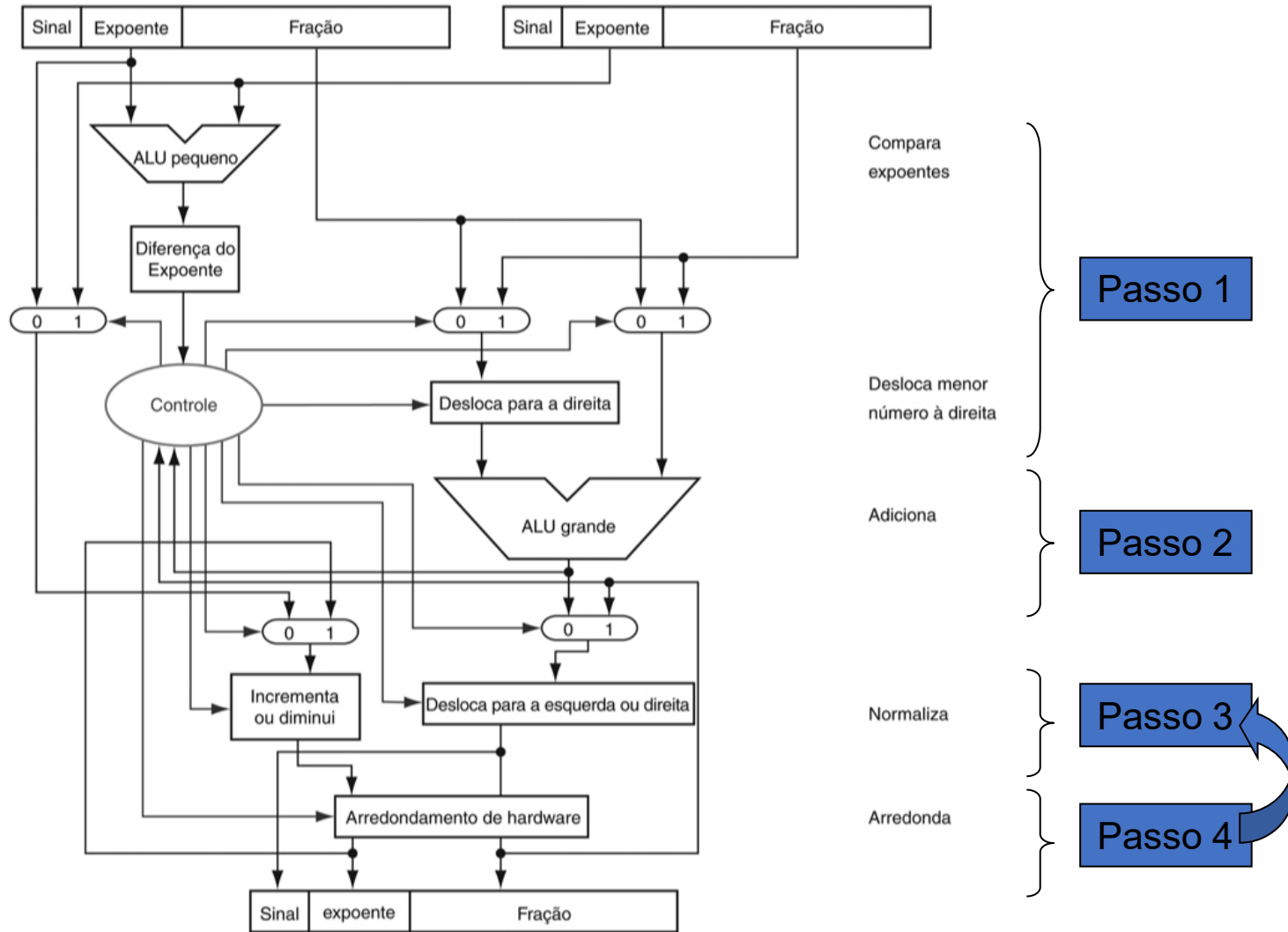
4. Arredondar e renormalizar se necessário

- $1.000_2 \times 2^{-4}$

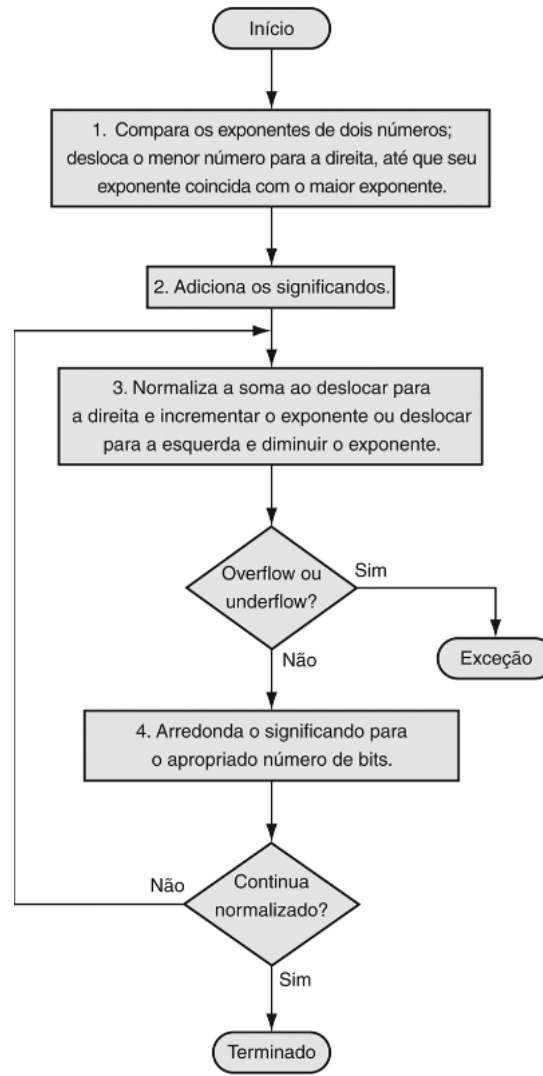
Adição: Hardware

- Muito mais complexo que o de adição de inteiros.
- Se realizada em um ciclo de clock somente, poderá trazer prejuízos
 - Muito maior que as operações com inteiros;
 - Diminuir a taxa de clock pode penalizar outras instruções;
- FP somador usualmente leva vários ciclos de clock para realizar sua tarefa.
 - Pode ser utilizada com pipeline (paralelizada).

Ponto Flutuante: HW Adição



Ponto Flutuante: HW Adição



Multiplicação: Hardware

- É similar ao somador, em complexidade
 - Usa multiplicador para as frações ao invés do somador
- O HW aritmético do FP usualmente realiza
 - Adição, subtração, multiplicação, divisão, radiciação...
 - Conversão FP $\leftrightarrow$ inteiro
- Usualmente leva vários ciclos de clock para realizar sua tarefa.
 - Pode ser utilizada com pipeline (paralelizada).

MIPS: Instruções em FP

- O HW é o coprocessador 1
 - Processador adjunto que estende a ISA
- Registradores de FP são separados
 - 32 single-precision: \$f0, \$f1, ... \$f31
 - Pareados para double-precision: \$f0/\$f1, \$f2/\$f3, ...
 - Release 2 do MIPS ISA (Instruction Set Architecture) suporta registradores de 32 × 64-bits
- Instruções de FP operam somente em registradores de FP
 - Programas geralmente não realizam operações em dados de FP ou vice-versa;
 - Mais registradores com mínimos impactos nos códigos.

MIPS: Instruções em FP

- A comparação em ponto flutuante define um bit como verdadeiro ou falso, dependendo da condição de comparação, e um desvio de ponto flutuante então decide se desviará ou não, dependendo da condição.
- Os projetistas do MIPS decidiram acrescentar registradores de ponto flutuante separados – chamados \$f0, \$f1, \$f2... — usados para precisão simples ou precisão dupla.
- Logo, eles incluíram loads e stores separados para registradores de ponto flutuante: lwc1 e swc1. Os registradores base para transferências de dados de ponto flutuante, usados para endereços, continuam sendo registradores inteiros.

MIPS: Instruções em FP

- Adição simples em ponto flutuante (add.s) e adição dupla (add.d)
- Subtração simples em ponto flutuante (sub.s) e subtração dupla (sub.d)
- Multiplicação simples em ponto flutuante (mul.s) e multiplicação dupla (mul.d)
- Divisão simples em ponto flutuante (div.s) e divisão dupla (div.d)
- Comparação simples em ponto flutuante (c.x.s) e comparação dupla (c.x.d), em que x pode ser igual (eq), diferente (neq), menor que (lt), menor ou igual (le), maior que (gt) ou maior ou igual (ge)
- Desvio verdadeiro em ponto flutuante (bc1t) e desvio falso (bc1f).

Exemplo: FP no MIPS

- C code: float f2c (float fahr)

```
{  
    return ((5.0/9.0)*(fahr - 32.0));  
}
```

- f2c: lwc1 \$f16, const5(\$gp)
 lwc2 \$f18, const9(\$gp)
 div.s \$f16, \$f16, \$f18
 lwc1 \$f18, const32(\$gp)
 sub.s \$f18, \$f12, \$f18
 mul.s \$f0, \$f16, \$f18
 jr \$ra

Lidando com o Overflow

- Algumas linguagens ignoram o Overflow (C)
 - Usam instruções MIPS: addu, addui, subu
- Outras linguagens (Fortran) necessitam de exceções
 - Usam instruções MIPS add, addi, sub
 - Quando houver o Overflow, invoca-se o Handler de exceções
 - Salve PC no registrador EPC - exception program counter
 - Jump para um endereço predefinido no Handler
 - A instrução mfc0 (move from coprocessor reg) pode recuperar o valor do EPC, para retornar depois de uma ação corretiva.

FP no MIPS

Operandos de ponto flutuante do MIPS

Nome	Exemplo	Comentários
32 registradores de ponto flutuante	\$f0, \$f1, \$f2, . . . , \$f31	Registradores MIPS de ponto flutuante são usados em pares para números de precisão dupla.
2 ³⁰ words em memória	Memória[0], Memória[4], . . . , Memória[4294967292]	Acessado apenas por instruções de transferência de dados. O MIPS usa endereços em bytes, de modo que os endereços sequenciais em palavras diferem em 4 vezes. A memória mantém estruturas de dados, como arrays, e spilled registers, como aqueles salvos em chamadas de procedimento.

Linguagem assembly de ponto flutuante do MIPS

Categoria	Instrução	Exemplo	Significado	Comentários
Aritmética	FP add single	add.s \$f2,\$f4,\$f6	\$f2 = \$f4 + \$f6	adição PF (precisão simples)
	FP subtract single	sub.s \$f2,\$f4,\$f6	\$f2 = \$f4 - \$f6	subtração PF (precisão simples)
	FP multiply single	mul.s \$f2,\$f4,\$f6	\$f2 = \$f4 × \$f6	multiplicação PF (precisão simples)
	FP divide single	div.s \$f2,\$f4,\$f6	\$f2 = \$f4 / \$f6	divisão PF (precisão simples)
	FP add double	add.d \$f2,\$f4,\$f6	\$f2 = \$f4 + \$f6	adição PF (precisão dupla)
	FP subtract double	sub.d \$f2,\$f4,\$f6	\$f2 = \$f4 - \$f6	subtração PF (precisão dupla)
	FP multiply double	mul.d \$f2,\$f4,\$f6	\$f2 = \$f4 × \$f6	multiplicação PF (precisão dupla)
Transferência de dados	FP divide double	div.d \$f2,\$f4,\$f6	\$f2 = \$f4 / \$f6	divisão PF (precisão dupla)
	load word copr. 1	lwc1 \$f1,100(\$s2)	\$f1 = Memória[\$s2 + 100]	dados de 32 bits para um registrador FP
Desvio condicional	store word copr. 1	swc1 \$f1,100(\$s2)	Memória[\$s2 + 100] = \$f1	dados de 32 bits para memória
	branch on FP true	bclt 25	if (cond == 1) go to PC + 4 + 100	desvio relativo ao PC se PF cond.
	branch on FP false	bclf 25	if (cond == 0) go to PC + 4 + 100	desvio relativo ao PC se não cond.
	FP compare single (eq,ne,lt,le,gt,ge)	c.lt.s \$f2,\$f4	if (\$f2 < \$f4) cond = 1; else cond = 0	comparação PF menor que, precisão simples
	FP compare double (eq,ne,lt,le,gt,ge)	c.lt.d \$f2,\$f4	if (\$f2 < \$f4) cond = 1; else cond = 0	comparação PF menor que, precisão dupla

FP no MIPS

Linguagem de máquina de ponto flutuante do MIPS

Nome	Formato	Exemplo						Comentários
add.s	R	17	16	6	4	2	0	add.s \$f2,\$f4,\$f6
sub.s	R	17	16	6	4	2	1	sub.s \$f2,\$f4,\$f6
mul.s	R	17	16	6	4	2	2	mul.s \$f2,\$f4,\$f6
div.s	R	17	16	6	4	2	3	div.s \$f2,\$f4,\$f6
add.d	R	17	17	6	4	2	0	add.d \$f2,\$f4,\$f6
sub.d	R	17	17	6	4	2	1	sub.d \$f2,\$f4,\$f6
mul.d	R	17	17	6	4	2	2	mul.d \$f2,\$f4,\$f6
div.d	R	17	17	6	4	2	3	div.d \$f2,\$f4,\$f6
lwc1	I	49	20	2	100			lwc1 \$f2,100(\$s4)
swc1	I	57	20	2	100			swc1 \$f2,100(\$s4)
bclt	I	17	8	1	25			bclt 25
bclf	I	17	8	0	25			bclf 25
c.lt.s	R	17	16	4	2	0	60	c.lt.s \$f2,\$f4
c.lt.d	R	17	17	4	2	0	60	c.lt.d \$f2,\$f4
Tamanho do campo		6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Todas as instruções MIPS de 32 bits